

1. Description courte du programme

Le projet consiste à réaliser un serveur Web configurable en Java.

Au démarrage, le serveur lit les fichiers de configuration XML placés dans `~/serveurWeb/conf.d`. Chaque fichier définit un site Web avec un port, un DocumentRoot, un fichier index par défaut et des fichiers de logs.

Pour chaque site configuré, le programme lance un thread contenant un ServerSocket. Cela permet de servir plusieurs sites en même temps sur plusieurs ports différents.

Quand un navigateur envoie une requête HTTP, le serveur lit la première ligne, récupère le chemin demandé, cherche le fichier correspondant dans le DocumentRoot, puis renvoie une réponse HTTP avec les bons en-têtes.

Le serveur gère aussi les erreurs 404 et 403, les logs, la page `/status`, le cache HTTP, la compression gzip, les listes de répertoires et l'exécution de code dynamique dans des pages HTML.

2. Arborescence finale

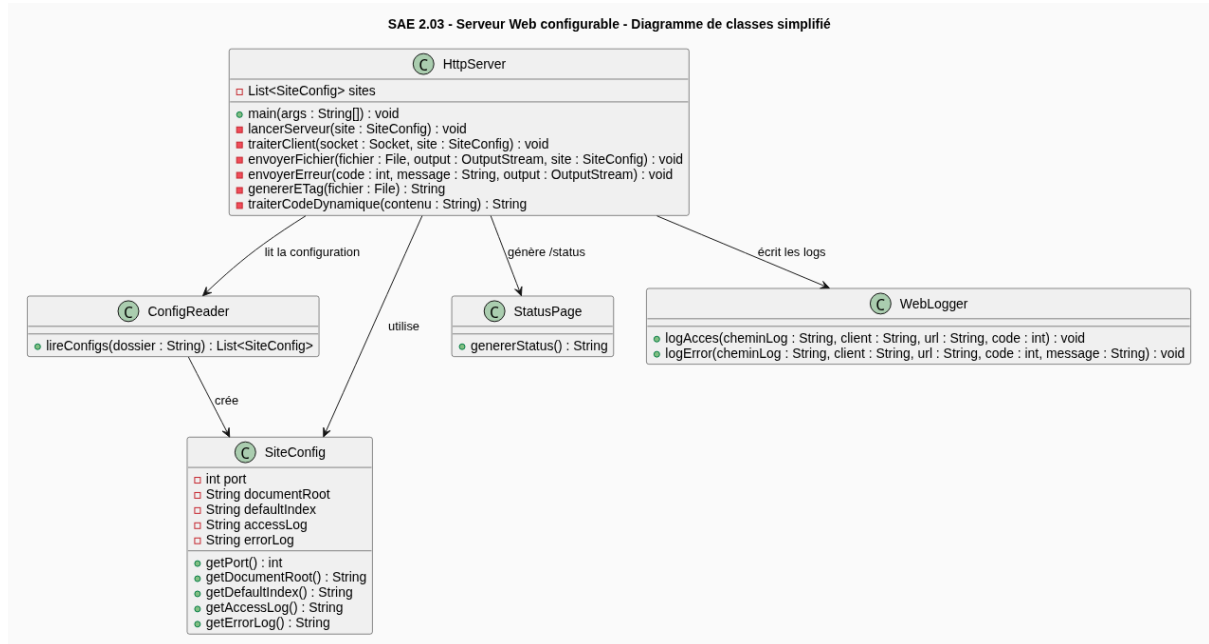
```
[Atlas] andalussi ~ > cd serveurWeb/  
[Atlas] andalussi ~/serveurWeb > tree
```

```
.  
├── bin  
│   ├── ConfigReader.class  
│   ├── HttpServer.class  
│   ├── SiteConfig.class  
│   ├── StatusPage.class  
│   └── WebLogger.class  
├── conf.d  
│   ├── serverWeb.conf  
│   └── site2.conf  
├── ConfigReader.java  
├── HttpServer.java  
├── logs  
│   ├── access.log  
│   └── error.log  
├── run  
├── scripts  
│   ├── start.sh  
│   └── stop.sh  
├── SiteConfig.java  
├── StatusPage.java  
├── WebLogger.java  
├── www  
│   ├── site1  
│   │   ├── dynamique.html  
│   │   ├── images  
│   │   │   ├── l1.gif  
│   │   │   └── n2.jpg  
│   │   ├── index.html  
│   │   ├── miniWeb.html  
│   │   └── technique.html  
│   └── site2  
│       └── index.html
```

```
10 directories, 23 files
```

```
[Atlas] andalussi ~/serveurWeb > █
```

3. Diagramme de classes PlantUML



4. Algorithmes principaux à expliquer

Lecture de configuration

1. Le serveur cherche les fichiers .conf dans ~/serveurWeb/conf.d.
2. Pour chaque fichier, il utilise une API XML.
3. Il récupère port, DocumentRoot, DefaultIndex, Acceslog et Errorlog.
4. Il crée un objet SiteConfig.
5. Le serveur lance un thread pour chaque SiteConfig.

Traitement d'une requête HTTP

1. Le serveur accepte une connexion TCP.
2. Il lit la première ligne HTTP.
3. Il récupère le chemin demandé.
4. Si le chemin vaut /, il utilise le fichier par défaut.
5. Il construit le chemin réel avec le DocumentRoot.
6. Il vérifie que le fichier reste dans le DocumentRoot.
7. Il renvoie le fichier, une liste de dossier, ou une erreur.

Sécurité 403

1. Le serveur calcule le chemin réel du DocumentRoot.
2. Il calcule le chemin réel du fichier demandé.
3. Il vérifie que le fichier demandé commence bien par le chemin du DocumentRoot.
4. Sinon, il renvoie 403 Forbidden.

Cache HTTP

1. Le serveur envoie un ETag et Last-Modified.
2. Le navigateur peut renvoyer If-Modified-Since.
3. Si le fichier n'a pas changé, le serveur répond 304 Not Modified.
4. Sinon, il renvoie le fichier normalement.

Gzip

1. Le serveur vérifie l'extension du fichier.
2. Si c'est png, jpg, jpeg ou pdf, il compresse le contenu.
3. Il ajoute Content-Encoding: gzip.
4. Il envoie le fichier compressé.

Code dynamique

1. Le serveur lit le HTML.
2. Il cherche les balises `<code interpreteur="...">...</code>`.
3. Il récupère l'interpréteur et le code.
4. Il exécute le code avec ProcessBuilder.
5. Il remplace la balise par le résultat.
6. Il envoie la page modifiée.

5. Exemple de fichier de configuration

```
<webconf>
  <site>
    <port>8080</port>
    <DocumentRoot>www/site1</DocumentRoot>
    <DefaultIndex>index.html</DefaultIndex>
    <Acceslog>logs/access.log</Acceslog>
    <Errorlog>logs/error.log</Errorlog>
  </site>
</webconf>
```

Deuxième site :

```
<webconf>
  <site>
    <port>8081</port>
    <DocumentRoot>www/site2</DocumentRoot>
    <DefaultIndex>index.html</DefaultIndex>
    <Acceslog>logs/access.log</Acceslog>
    <Errorlog>logs/error.log</Errorlog>
  </site>
</webconf>
```

6. Scripts Linux

start.sh

```
#!/bin/bash
```

```
cd ~/serveurWeb
```

```
java -cp ~/serveurWeb/bin HttpServer
```

stop.sh

```
#!/bin/bash
```

```
PID_FILE=~/.serveurWeb-/run/myweb.pid
```

```
if [ -f "$PID_FILE" ]; then
    kill $(cat "$PID_FILE")
    rm "$PID_FILE"
    echo "Serveur arrêté"
else
    echo "Aucun fichier PID trouvé"
fi
```

7. Service systemd utilisateur

[Unit]

Description=Serveur Web Java configurable

After=network.target

[Service]

Type=simple

ExecStart=%h/serveurWeb/scripts/start.sh

ExecStop=%h/serveurWeb/scripts/stop.sh

WorkingDirectory=%h/serveurWeb

Restart=on-failure

[Install]

WantedBy=default.target

Commandes :

```
systemctl --user daemon-reload
```

```
systemctl --user start serverWeb
```

```
systemctl --user status serverWeb
```

```
systemctl --user stop serverWeb
```

8. Package Debian

```
[Atlas] andalussi ~/package-serverWeb > tree
```

```
.
├── DEBIAN
│   └── control
├── home
│   └── andalussi
│       └── serveurWeb
│           ├── bin
│           │   ├── ConfigReader.class
│           │   ├── HttpServer.class
│           │   ├── SiteConfig.class
│           │   ├── StatusPage.class
│           │   └── WebLogger.class
│           ├── conf.d
│           │   ├── serverWeb.conf
│           │   └── site2.conf
│           ├── ConfigReader.java
│           ├── HttpServer.java
│           ├── logs
│           │   ├── access.log
│           │   └── error.log
│           ├── run
│           ├── scripts
│           │   ├── start.sh
│           │   └── stop.sh
│           ├── SiteConfig.java
│           ├── StatusPage.java
│           ├── WebLogger.java
│           └── www
│               ├── site1
│               │   ├── dynamique.html
│               │   ├── images
│               │   │   ├── l1.gif
│               │   │   └── n2.jpg
│               │   ├── index.html
│               │   ├── miniWeb.html
│               │   └── technique.html
│               └── site2
│                   └── index.html
```

```
14 directories, 24 files
```

```
[Atlas] andalussi ~/package-serverWeb > █
```

Package: serverweb

Version: 1.0

Section: web

Priority: optional

Architecture: all

Maintainer: Zayd

Depends: default-jre

Description: Serveur Web Java configurable

Serveur web simple en Java avec configuration XML, logs, cache, gzip, page status, code dynamique, scripts Linux et service systemd utilisateur.

Commande de construction :

```
dpkg-deb --root-owner-group --build package-serverweb/
```

Commande pour voir le contenu :

```
dpkg-deb -c package-serverweb.deb
```

9. Tests

```
curl -I http://localhost:8080/
```

```
curl -I http://localhost:8081/
```

Vérifie plusieurs sites.

```
curl -I http://localhost:8080/inexistant.html
```

Vérifie 404.

```
curl -I --path-as-is "http://localhost:8080/../../../../etc/passwd"
```

Vérifie 403.

```
curl -I http://localhost:8080/images/n2.jpg
```

Vérifie gzip.

```
curl -I http://localhost:8080/
```

```
curl -I -H "If-Modified-Since: DATE_RECUPEREE" http://localhost:8080/
```

Vérifie cache 304.

```
curl http://localhost:8080/status
```

Vérifie status.

```
curl http://localhost:8080/dynamique.html
```

Vérifie code dynamique.

```
cat ~/serveurWeb/logs/access.log
```

```
cat ~/serveurWeb/logs/error.log
```

Vérifie logs.

10. Conclusion

Ce projet nous a permis de comprendre le fonctionnement de base d'un serveur Web. Le serveur utilise des sockets TCP pour recevoir les requêtes d'un navigateur, analyse les requêtes HTTP, retrouve les fichiers demandés et renvoie une réponse HTTP correcte.

La partie configurable permet de modifier les ports, le DocumentRoot et les logs sans recompiler.

Le serveur peut aussi gérer plusieurs sites en parallèle grâce à des threads.

Enfin, le projet est intégré à Linux avec un fichier PID, des scripts de démarrage et d'arrêt, un service systemd utilisateur et un package Debian.